AI-Based Restoration of Degraded Images

A complete account of the project, from problem to solution

KLA Challenge · i4C Hackathon 2026 · solo entry
Joint 2× super-resolution and speckle denoising for semiconductor wafer inspection imagery

Who this document is for

This assumes **no background** in image processing or machine learning. Every technical term is explained the first time it appears, and there is a glossary at the end.

It is organised as a story with four movements: what the problem was, what solutions existed, which one we chose and why, and how it actually works inside. Every number quoted was measured on this project, not taken from a paper.

Contents

1. The problem

1.1 Where these images come from

Computer chips are manufactured on thin discs of silicon called **wafers**. A single wafer carries hundreds of chips, and the features printed on it are measured in nanometres — thousands of times thinner than a human hair. At that scale a speck of dust or a malformed edge ruins a chip entirely.

KLA builds the inspection machines that photograph these wafers looking for such defects. Those machines produce enormous numbers of images, and the quality of each image directly determines whether a defect is caught or missed. This is the context: the images in this challenge are not holiday photos, they are measurement data, and detail in them carries real consequences.

1.2 The challenge, in one sentence

> **The task**
>
> You are given an image that has been **shrunk to half size** and had **noise added**. You must produce the original: full size and clean.

Formally this is two classic problems welded together. Making a small image larger is called **super-resolution**. Removing noise is called **denoising**. Doing both at once is harder than either alone, because the two goals fight each other: the safest way to remove noise is to blur the image, and blurring destroys exactly the fine detail that super-resolution is supposed to recover.

1.3 Why this is genuinely hard

The central difficulty is that **information was permanently destroyed**. When an image is halved in size, each square of four pixels is averaged down into one. Four numbers became one number. Nothing can recover the original four, in the same way that knowing two numbers average to 10 does not tell you whether they were 9 and 11 or 3 and 17.

So no algorithm 'undoes' the shrinking. What a good algorithm does instead is produce the most **plausible** reconstruction — it has seen a great many images of this kind, so when it encounters a particular grey smudge it knows what such a smudge usually looks like at full resolution, and draws that in.

This distinction matters throughout the document. The noise removal is genuine recovery. The enlargement is educated invention.

1.4 How the work is judged

Submissions are scored on two completely separate axes, and this shapes almost every decision in the project.

Axis	What it measures	How
Restoration quality	How close the output is to the true original image	Three metrics: pSNR, SSIM and LPIPS (all explained in section 8.1)

Axis	What it measures	How
End-to-end speed	Total wall-clock time on an NVIDIA H100 GPU	Measured from the moment the program launches: start-up, loading the model, reading files from disk, processing, and writing results back

> **The consequence that shapes everything**
>
> Because the clock starts when the **program launches**, not when processing begins, ordinary optimisation advice inverts. Several standard techniques that make processing faster do so by spending time up front preparing — and that preparation is on the clock. Section 7 shows a case where the textbook optimisation made the program **47% slower**.

1.5 The hard constraints

- **Training happens on a laptop.** An RTX 4060 laptop GPU with 8 GB of memory, which thermally throttles under sustained load. This rules out most of the highest-quality published architectures outright — they simply will not fit.
- **Judging happens on an H100**, a datacentre GPU roughly an order of magnitude more capable. So the model must be *trainable* on modest hardware while being *fast* on powerful hardware.
- **One model must handle two different sizes.** The test set contains images to be doubled from 128×128 to 256×256, and others from 256×256 to 512×512. A single set of weights must serve both.
- **The evaluation script must run unmodified.** It receives two arguments — an input folder and an output folder — and must work with no manual intervention. If it crashes on an unexpected input, that portion of the test set simply scores nothing.
- **Generalisation is explicitly scored.** The test set deliberately includes images from sources unlike anything in training.

1.6 The complication that defined the project

Challenges of this kind normally supply a training set: pairs of damaged and original images to learn from. **No such data was ever released.** Submission was direct.

This turns a standard supervised learning task into something much less standard. With no examples of the damage, we could not simply learn to reverse it. We had to first work out precisely how the organisers damaged their images, then manufacture our own training examples by damaging clean images the same way.

The problem statement explicitly permits this — it states that synthetic data generation is encouraged — but it makes the accuracy of our reverse-engineering the foundation everything else rests on. Section 3 is devoted to it.

2. Understanding the damage

Before anything can be repaired it has to be understood. This section explains each of the three things done to the images, and why one of them behaves very differently from what most people expect of 'noise'.

2.1 Damage one: shrinking

The image is reduced to half its width and half its height. The standard way to do this is to take each square block of 2×2 pixels and replace it with their average.

```
original 4x4 block            halved 2x2 result

   10  20 | 30  40            (10+20+50+60)/4 = 35   ...
   50  60 | 70  80      ->             35               55
   ------+------                      75               95
   70  80 | 90 100
   80  90 |100 110
```

Four pixels collapse into one. Three quarters of the numbers are gone.

A quarter of the data survives. This is why the enlargement half of the task is invention rather than recovery.

> **A useful thing we verified**
>
> Image libraries offer several named methods for shrinking — `area`, `bilinear`, `bicubic`, `lanczos`, `nearest`. We measured that at an exact 2× reduction, `area` **and** `bilinear` **produce mathematically identical results**, and both are exactly the simple 2×2 block average shown above. The difference between them measured **exactly 0.0** against the block average, and 0.00000006 for bilinear, which is floating-point rounding rather than a real difference.
>
> At a non-integer reduction such as 512→300 the same two methods differ by 0.44, so this equivalence is specific to the exact-halving case — which is the only case this problem contains.

2.2 Damage two: speckle, the unusual noise

Most people picture noise as television static: random flecks scattered evenly across the picture, the same amount everywhere. That is **additive** noise — a random amount is added to each pixel, independent of how bright that pixel was.

The dominant noise here is not that. It is **multiplicative**, meaning each pixel is *multiplied* by a random factor rather than having something added to it. The practical effect is that the corruption is a **percentage error**:

True brightness	Additive noise (±0.05)	Multiplicative noise (±24%)
0.10	0.05 – 0.15	0.076 – 0.124
0.50	0.45 – 0.55	0.38 – 0.62
1.00	0.95 – 1.05	**0.76 – 1.24**

With multiplicative noise the damage grows with brightness. Dark regions are barely touched; bright regions are wrecked.

Why this kind of noise exists physically

Speckle appears whenever an image is formed using **coherent** light or waves — laser illumination, radar, ultrasound, and several kinds of electron microscopy. Coherent waves reflecting off a slightly rough surface arrive at the detector having travelled marginally different distances, so they interfere with one another, sometimes reinforcing and sometimes cancelling. The result is a grainy pattern whose strength is proportional to the underlying signal. It is an inherent property of the imaging physics, not a fault in the equipment.

The consequence that gives the problem away

Because the corruption multiplies, a bright pixel can be pushed **above the maximum possible brightness**. If image brightness runs from 0 (black) to 1 (white), and a pixel that should be 1.0 gets multiplied by 1.24, the result is 1.24 — brighter than white.

The problem statement flags this explicitly as expected behaviour. It is a strong clue, and it is the reason our pipeline never clips input values: the out-of-range portion is real information about the noise, and discarding it would throw away signal.

2.3 Measuring speckle strength: the number of looks

Speckle strength is described by a single number conventionally written **L**, called the *number of looks*. The name comes from radar, where averaging several independent observations (looks) of the same scene reduces the graininess.

The rule is simple: the relative strength of the speckle is one divided by the square root of L.

```
speckle strength  =  1 / sqrt(L)

   L = 4    ->  50%   relative noise   (severe)
   L = 17   ->  24%   relative noise   (what this challenge uses)
   L = 100  ->  10%   relative noise   (mild)
```

Mathematically the multiplier is drawn from a **Gamma distribution** with mean exactly 1 and variance 1/L. Mean 1 is what makes it noise rather than a brightness change — on average it leaves the image alone, it just scatters individual pixels around their true values.

2.4 Damage three: a small additive noise

On top of the speckle there is a second, much weaker noise of the ordinary additive kind — the television-static sort, the same magnitude everywhere regardless of brightness. Its strength is written **σ** (sigma), the standard statistical symbol for spread.

In this challenge σ sits somewhere around 0.001 to 0.009, against speckle contributing roughly 0.24. It is therefore **25 to 200 times weaker** than the speckle and is very much a minor character — but it exists, and it matters for one specific reason explained in section 3.5.

2.5 The complete recipe

```
ORIGINAL IMAGE  (512 x 512, brightness 0 to 1)
       |
       |   average each 2x2 block of pixels
       v
HALF-SIZE IMAGE (256 x 256, still clean)
       |
       |   multiply every pixel by a random factor
       |   (Gamma distributed, average 1, spread 1/sqrt(L))
       v
SPECKLED IMAGE
       |
       |   add a small random amount to every pixel
       |   (Gaussian, spread sigma)
       v
DEGRADED IMAGE  (256 x 256, brightness may exceed 1)
```

The full forward process. Our model must run this backwards.

2.6 An ambiguity in the problem statement

The official slides are not internally consistent about the damage. Four separate slides list **two** kinds of damage — speckle and shrinking. One slide lists **three**, adding an item spelled 'Gausian noise' and described as *'reduction in image sharpness and edge detail'*.

That description is a description of **blur**, not of noise. Blur and noise are opposites in an important sense: blur removes detail, noise adds spurious detail. Building the wrong one into our training data would have been a serious error. Section 3.4 explains how the ambiguity was resolved.

3. The detective work

With no training data, everything depended on discovering the organisers' exact recipe. This section describes how that was done, what was found, and how it was checked.

3.1 Why the recipe had to be exact

The plan was to manufacture training data: take clean images, damage them ourselves, and let the model learn to reverse our damage. That only works if our damage closely matches theirs. A model trained to remove 10% noise performs poorly on 24% noise; a model trained to reverse one kind of shrinking is not reversing the same operation as another.

In short: get the recipe wrong and you have spent your entire effort practising for the wrong examination.

3.2 Where the parameters were hiding

The problem statement was distributed as a PowerPoint file. A `.pptx` file is not really a single document — it is a compressed archive containing separate files for each slide's text and each embedded picture. Unpacking it gives direct access to all of them.

Doing that revealed that the two sample figures were not screenshots. They were charts generated programmatically by the organisers' own code, and the code had written the settings used into the chart titles:

```
Training - Sample 000000
Source: 0001.png | L=16.86, sigma=0.008594

Training - Sample 000500
Source: 0186.png | L=18.13, sigma=0.001065
```

Read at 4x magnification to confirm every digit. These two lines are the only quantitative evidence about the degradation in existence.

3.3 What the numbers mean

Symbol	Observed	Meaning	Implication
L	16.86 and 18.13	Speckle look-count	Relative noise of 1/√17 ≈ **24%** — the dominant damage
σ	0.008594 and 0.001065	Additive noise spread	**25–200× weaker** than speckle; secondary

Two observations from just two samples. The L values differ by 7%; the σ values differ by a factor of eight. That asymmetry suggests L is drawn from a narrow range and σ from a wide one — but two data points cannot establish a distribution, and we were careful to treat this as a hint rather than a finding.

3.4 Resolving the blur ambiguity

Section 2.6 left open whether the third damage was blur or additive noise. The captions settle it, by a simple argument about units.

If σ described **blur**, it would be a width measured in pixels. A blur of 0.001 pixels is not a small blur — it is nothing at all. Blurring by a thousandth of a pixel on a grid of whole pixels has no effect whatsoever; the operation is mathematically an identity.

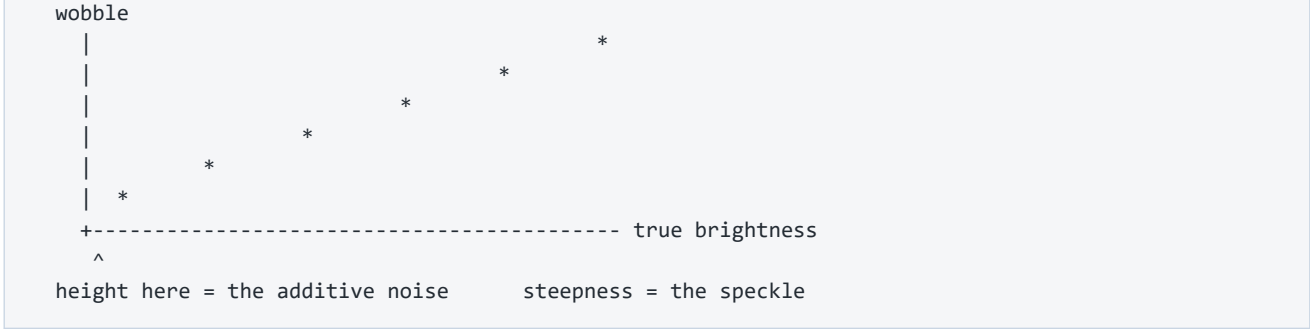
If σ describes **additive noise**, it is a brightness spread on a scale from 0 to 1. Values of 0.001 and 0.0086 are then small but entirely meaningful.

> **Conclusion**
>
> Only one reading makes the numbers meaningful, so the third damage is **additive Gaussian noise**, and the slide's wording is an error in the problem statement.
>
> Supporting evidence: only one of five slides mentions it, and that slide misspells 'Gaussian'.

3.5 Extracting both noise strengths from one measurement

Suppose we did have real damaged/clean pairs. Could we measure L and σ from them? Yes — and the method is worth explaining because it is elegant and because it was used to verify our own tools.

The two noises look identical in any single pixel. But they behave differently as brightness changes: speckle grows with brightness, additive noise does not. So sort every pixel by how bright it *should* be, group them into buckets, and measure how much each bucket wobbles.



One straight line yields both answers: its steepness gives the speckle strength, and where it starts on the left gives the additive strength. There is a bonus — if the points do *not* form a straight line, the whole theory of how the damage works is wrong. On our test data they formed a line to **99.98%** accuracy.

3.6 Proving the tools work before trusting them

Here is the difficulty with detective work when there is no real evidence: how do you know your methods are sound?

The answer used throughout this project was to **plant answers and see if we could find them**. We generated fake damaged images using settings we chose ourselves, then ran the measuring code on them and checked whether it recovered the numbers we had picked. If it cannot find an answer we deliberately hid, nothing it says about real data is worth reading.

Quantity	Value we planted	Value recovered	Error
Shrinking method	block average	correct (as a tied pair)	—

Quantity	Value we planted	Value recovered	Error
Speckle strength L	17.053	16.817	**1.4%**
Additive σ	0.006279	0.006266	**0.2%**

Verification of the measurement code against known ground truth.

3.7 Three things this exercise taught us

(a) The obvious approach to identifying the shrinking method was wrong

The natural instinct is to blur both images first to wash out the noise, then compare. We measured that this reduced the gap between the correct answer and the wrong ones to **0.0%** — the test could no longer identify an answer we had handed it.

The reason: the shrinking methods differ almost entirely in how they treat the finest details, and blurring destroys exactly those details first. It is like trying to identify someone's handwriting by squinting until the page is a grey smudge.

The correct approach is to compare without blurring at all. The noise makes every candidate look equally bad — it is a headwind every runner faces, so the ranking still holds. Without blurring, the correct answer won on **24 out of 24** test images.

(b) The additive noise is hard to measure the obvious way

Reading 'where the line starts' by extending it backwards is unreliable when all your measurements sit far from that point — a tiny error in the tilt swings the starting point wildly. Our first attempt came out **35% too low** and on individual images frequently produced impossible negative values.

The fix was to stop extrapolating and measure directly: look only at the **darkest** pixels, where speckle barely exists, so whatever wobble remains must be the additive noise. Error fell from 35% to **0.2%**.

(c) Some questions have no answer, and that is fine

As noted in section 2.1, two of the candidate shrinking methods are the same operation at exactly 2×. No amount of cleverness can distinguish them, because there is nothing to distinguish. Our first version of the test reported FAIL for 'guessing wrong' between two identical things — the test was wrong, not the method. It now reports a group of equivalent answers instead of pretending to a precision that does not exist.

3.8 Confirming it visually

Finally, we generated damaged images using the two exact settings from the captions and compared them against the organisers' own published figures.

Sample	Their figure reaches	Ours reaches	Verdict
Texture sample	≈ 1.1 – 1.2	**1.196**	match
Dendrite sample	≈ 1.6	**1.773**	same regime

Peak brightness of the damaged image, where the original maxes out at 1.0. The overflow behaviour and histogram shapes both reproduce.

This is the strongest available confirmation. It is not proof — only real paired data would be proof — but the reconstructed recipe reproduces the published behaviour closely, including the distinctive overflow past maximum brightness.

4. Possible solutions

There were many ways to attack this. This section lays out the realistic options and the reasoning that eliminated each one, because the choice only makes sense against the alternatives.

4.1 Approaches that use no learning at all

Classical image processing offers a toolkit that requires no training data and no GPU. These were not seriously candidates for the final submission, but they matter enormously as **baselines** — the floor any learned model must clear to justify its existence.

Method	How it works	Why not sufficient
Bicubic interpolation	Fills in new pixels by fitting smooth curves through neighbours	Enlarges but cannot denoise. Produces soft, blurry output
Median filter	Replaces each pixel with the middle value of its neighbours	Good against outliers, but erases fine detail along with the noise
Bilateral filter	Averages neighbours, but only those of similar brightness, so edges survive	Better edge preservation, still no ability to invent lost detail
Non-local means / BM3D	Finds similar patches elsewhere in the image and averages them	Strong denoiser, genuinely good, but slow and still no super-resolution
Lee / Kuan / Frost filters	Designed specifically for multiplicative speckle in radar imagery	Correct noise model, but purpose-built for denoising only
Wavelet thresholding	Decomposes into frequency bands and suppresses small coefficients	Fast, but tuned per image and cannot restore resolution

The common thread: none of these can **invent** plausible detail, because none of them has ever seen what this kind of imagery looks like at full resolution. They can only rearrange what is already present. That ceiling is precisely why learned methods win here.

4.2 Approaches that learn

A neural network trained on many examples can do what the classical methods cannot: it builds an internal sense of what this category of image looks like, and uses that to fill in what was destroyed. The question was which architecture.

Architecture	Strengths	Weaknesses	Verdict
Plain CNN (SRCNN, EDSR)	Simple, fast, easy to train	Limited quality ceiling; no multi-scale view of the image	Too weak

Architecture	Strengths	Weaknesses	Verdict
U-Net	Sees the image at several scales at once; small memory footprint; very hard to get wrong	Lower ceiling than modern designs	**Built as our baseline**
NAFNet	Deliberately stripped down: no attention, no costly activation functions. Strong published results on denoising and deblurring at low memory cost	Slightly less capable on very long-range structure	**Chosen**
Restormer	Transformer with channel attention; excellent quality	Memory hungry; painful to train in 8 GB	Rejected: will not fit
SwinIR	Among the best published super-resolution quality	Window attention is slow at inference — directly penalised by the speed score	Rejected: too slow
HAT	State of the art on super-resolution benchmarks	Heavier still than SwinIR	Rejected: same reasons, more so
GAN (Real-ESRGAN)	Sharpest, most convincing-looking output	Notoriously unstable to train; needs a second network and careful balancing; can invent detail that is not there	Rejected: too risky solo
Diffusion (SR3, ResShift)	Best perceptual quality currently achievable	Runs the network dozens of times per image	Rejected: disqualifying on speed

The two constraints that decided it

Training memory, not inference speed, was the binding constraint. An 8 GB laptop GPU eliminates the transformer family before quality is even discussed.

Speed is scored. That eliminates the diffusion family, which is otherwise the quality leader, because running a network thirty times per image cannot compete with running it once.

What remains is the efficient-CNN family, and within it NAFNet is the strongest well-documented option.

5. Our solution

5.1 The shape of the whole system

```
CLEAN IMAGES from three sources (10,506 total)
      |
      |  pick one at random, cut out a 256x256 patch
      |  flip / rotate it at random
      v
TARGET  (what the model must produce)
      |
      |  DAMAGE IT OURSELVES, with fresh random settings each time:
      |    - pick a shrinking method at random
      |    - pick a speckle strength L at random from 8 to 40
      |    - pick an additive sigma at random from 0.0005 to 0.02
      v
INPUT  (128x128, damaged)
      |
      |  NAFNet, 11 million parameters
      v
PREDICTION (256x256)   -- compared against TARGET, three ways
      |
      |  adjust all 11 million parameters slightly
      v
repeat 48,000 times
```

5.2 The single most important decision: manufacture the data

With no supplied pairs, every training example is made by us. This sounds like a handicap. In one important respect it is an advantage.

A model trained on a fixed dataset sees each damaged image exactly as it was provided, and can slowly memorise those specific images. Our model sees a **freshly damaged version every single time**, with a newly drawn noise strength and shrinking method. It never encounters the same corruption twice, so there is nothing to memorise. The only way to reduce the error is to actually learn how to reverse damage in general.

Choosing the ranges when you cannot measure the truth

The captions gave two speckle values, 16.86 and 18.13. The obvious move is to train at exactly that level. That would be a mistake.

Those two numbers are a sample of two. The real generator may draw from a much wider range, and the test set explicitly contains material from different sources. If we train narrowly at L = 17 and the truth is L = 8, performance collapses.

Setting	Observed	Trained over	Reasoning
Speckle L	16.86, 18.13	**5 to 60**	Started at 8–40; widened after the robustness sweep in section 8.5 found the model weak below L = 8
Additive σ	0.001065, 0.008594	**0.0005 to 0.03**	Sampled on a log scale, since the two observations differ 8-fold

Setting	Observed	Trained over	Reasoning
Shrinking method	unknown	**all four**	We cannot identify it, so we cover every candidate

> **The principle**
>
> **A range that contains the truth beats a point estimate that misses it.** We accept slightly lower performance at any single setting in exchange for not collapsing at the real one.

5.3 Where the clean images came from

Everything the model knows about what images look like comes from this corpus, so its composition is the single biggest quality lever available.

Source	Count	What it contributes
EPFL electron microscopy	3,367	Real microscope imagery — the closest publicly available analogue to wafer inspection data
Describable Textures Dataset	5,639	47 families of texture. Structural variety, which is what the unseen-sources half of the test set probes
Procedurally generated	1,500	Written by us: dendrites, granular fields, grain boundaries, repeating die lattices, fibres
	10,506	total

Why generate images by hand at all

Neither downloaded corpus is semiconductor imagery. Super-resolution has to learn what plausible fine detail looks like **for this specific domain** — a model that has only seen photographs will hallucinate photograph-like texture into a wafer image.

The generated set fills that gap directly. It reproduces the two structure types actually visible in the organisers' sample figures (a granular texture and a branching dendrite) and adds three more that inspection imagery is full of. The repeating **lattice** pattern — rows of identical chips, line-and-space structures, scattered contact holes — has no analogue in either downloaded corpus and is arguably the most characteristic structure in real wafer images.

5.4 Validation that is actually honest

A held-out set of 525 images is kept entirely out of training. Two details make the resulting numbers trustworthy:

- **Whole images are held out, not patches.** If patches from the same image appeared in both sets, the model could score well by recognising material it had already seen.
- **The split is balanced across all three sources**, so validation cannot end up dominated by whichever corpus happens to be easiest.
- **Validation damage is deterministic.** The same noise is drawn every epoch, so when the score moves it reflects the model changing, not the dice.

6. How it works, in depth

This section opens the box. It builds up from the individual operations to the complete network.

6.1 What a convolution is

The basic operation is the **convolution**. Slide a small window — typically 3×3 pixels — across the image. At each position, multiply the nine pixels under the window by nine stored numbers and add up the result. That single number becomes one pixel of the output.

Those nine stored numbers are what the network *learns*. Different sets detect different things: one might respond to vertical edges, another to a particular texture. A layer applies dozens of such windows in parallel, producing dozens of output images called **channels** or feature maps, each highlighting something different.

```
input 1 channel        ->  first layer  ->  48 channels
(the grey image)           (48 filters)     (48 different 'views')
```

6.2 Seeing the image at several scales

A 3×3 window only sees three pixels across. To recognise a structure spanning 50 pixels you must either stack a great many layers, or shrink the image so that 50 pixels becomes 6.

The second approach is the **encoder–decoder** (or U-Net) design used here. The image is progressively halved while the channel count doubles, processed at the smallest scale where each window covers a large real area, then progressively enlarged back.

```
128x128, 48ch  -------------------------->  128x128, 48ch
      |  halve                          enlarge  ^
      v                                          |
 64x64, 96ch   -------------------------->   64x64, 96ch
      |  halve                          enlarge  ^
      v                                          |
 32x32, 192ch  -------------------------->  32x32, 192ch
      |  halve                          enlarge  ^
      v                                          |
 16x16, 384ch  ---- 8 processing blocks ---------+

the horizontal arrows are SKIP CONNECTIONS
```

The horizontal **skip connections** are essential. Shrinking loses fine detail; the skips carry the high-resolution information directly across so the enlarging half has both the big-picture understanding from the depths and the fine detail from the surface.

6.3 Inside a NAFNet block

NAFNet's contribution is what happens at each processing step. Its name stands for **Nonlinear Activation Free** network, and the point is what it removes.

Why networks need a nonlinearity at all

Stacking convolutions alone is pointless: a chain of linear operations collapses mathematically into a single linear operation, so a hundred layers would be no more capable than one. Something non-linear must sit between them. Traditionally this is a function like ReLU (negatives become zero) or GELU (a smoother variant).

SimpleGate: multiplication instead of a function

NAFNet discards those functions entirely. Instead it takes its stack of channels, **splits it into two halves, and multiplies them together element by element**.

```
input:  96 channels
        |
        +-- first 48 channels  ---+
        |                         x   (multiply, position by position)
        +-- second 48 channels ---+
        |
output: 48 channels
```

Multiplication is non-linear, so it does the required job. It is also cheaper than computing a GELU, and it gives the network a useful ability: one half can act as a **gate** on the other, learning to suppress or amplify the second half depending on context.

Simplified Channel Attention: letting the whole image vote

Each channel detects something different. Which are useful depends on the image — an edge detector matters in a structured region and not in flat background.

So the block averages each channel down to a **single number** summarising it across the whole image, passes those through a small learned layer, and uses the result to scale each channel up or down. Cheap, and it lets the network modulate behaviour based on overall image content.

The word *simplified* is doing real work. Full attention, as used in transformers, compares every position against every other position — cost grows with the square of the pixel count, which is what makes those models unaffordable here. This version compares nothing; it just pools and scales.

The complete block

```
x -->[normalise]-->[expand]-->[depthwise 3x3]-->[SimpleGate]
                                                    |
                                          [channel attention]
                                                    |
                                                [compress]
                                                    |
x --------------------------(+ beta * ...)----------+
|
+-->[normalise]-->[expand]-->[SimpleGate]-->[compress]
|                                                   |
+--------------------------(+ gamma * ...)-----+
```

Two details worth noting. **Depthwise** convolution applies one filter per channel rather than mixing all channels, which is far cheaper. And **beta** and **gamma** are learned scaling numbers that both start at **zero** — so at the start of training every block does nothing at all, passing its input straight through. The network

begins as an identity function and each block gradually learns how much to contribute. This is what makes deep stacks trainable without elaborate warm-up tricks.

6.4 Enlarging: PixelShuffle

At the end the image must be doubled. The naive approach is transposed convolution, which is prone to a characteristic **checkerboard** artifact when its overlapping windows do not tile evenly.

PixelShuffle avoids the problem entirely by not computing anything. The network produces four channels at the small size; PixelShuffle rearranges each group of four values into a 2×2 square of output pixels. It is a pure reshuffle — no arithmetic, so no artifacts.

```
4 channels at 128x128        ->      1 channel at 256x256

   ch0[y,x] ch1[y,x]                   out[2y,   2x  ] = ch0[y,x]
   ch2[y,x] ch3[y,x]        ==>        out[2y,   2x+1] = ch1[y,x]
                                       out[2y+1, 2x  ] = ch2[y,x]
                                       out[2y+1, 2x+1] = ch3[y,x]
```

6.5 Two design decisions that matter more than the architecture

(a) Do all the work small, enlarge exactly once at the very end

The network could enlarge first and then clean up at full size. Doing the opposite — processing entirely at the small size and enlarging in the final step — costs **four times less** computation and memory, since doubling both dimensions quadruples the pixel count.

Is it also *better*, not merely cheaper? We tested the idea independently using simple non-learned filters, and the answer turned out to be more mixed than first appeared:

Approach	pSNR	SSIM	LPIPS
Enlarge first, then denoise	21.395	**0.5560**	**0.4189**
Denoise first, then enlarge	**21.802**	0.5214	0.4352
difference	**+0.407**	−0.0346	+0.0163

Measured on 400 validation samples. Denoising first wins clearly on pixel accuracy but loses on the structural and perceptual metrics.

> **A correction worth recording**
>
> An earlier measurement on only 150 samples showed denoise-first winning by **1.0 dB** and on LPIPS as well, and that was written up as clean independent support for the architecture. Re-measuring on 400 samples cut the pSNR gap to **0.407 dB** and reversed the SSIM and LPIPS results entirely.
>
> The original sample was simply too small for a difference that size. The architectural choice still stands on the compute argument, which is exact and not statistical — but it is *not* cleanly supported by this experiment, and saying otherwise would have been overclaiming.

(b) Predict the correction, not the answer

The final output is not produced from scratch. A plain bilinear enlargement of the input is computed and **added** to the network's output, so the network only has to learn the difference.

This works because speckle averages out to no change — a simple enlargement is already a reasonable first guess. The last layer is initialised to **exactly zero**, so an untrained network outputs precisely the bilinear enlargement and nothing else. We verified this: the difference measured **0.00e+00**, exactly zero, not approximately.

The network therefore starts from a sensible answer and spends all its capacity on improvement, never wasting effort learning to reproduce an operation that is free.

6.6 What the model is scored against while learning

Three separate measures are combined, weighted 1.0, 0.2 and 0.1:

Term	What it measures	Why it is included
Charbonnier	Pixel-by-pixel difference, in a form that does not over-react to extreme outliers	Speckle produces genuine extreme pixels. Standard squared error would let a handful of them dominate the learning signal
SSIM	Structural similarity — whether patterns and local contrast match	One of the three scored competition metrics
LPIPS	Perceptual similarity, judged by a separate pre-trained network	Also a scored metric; captures 'looks right to a human' in a way pixel arithmetic does not

> **Why not just optimise pixel accuracy**
>
> Optimising pixel error alone reliably produces **blurry** output. A blur is never badly wrong anywhere, so it is the safe choice under pixel-wise scoring — it hedges. The structural and perceptual terms punish hedging and force the network to commit to plausible detail.

6.7 How the training run is set up

Setting	Value	Reason
Optimiser	AdamW	Standard, robust, adapts step size per parameter
Learning-rate schedule	One-cycle, peak 0.0002	Warms up gently, then anneals to near zero, which typically buys a final improvement as it settles
Precision	Mixed (fp16 + fp32)	Roughly doubles throughput. Delicate steps stay in full precision
Batch size	12 (auto-reduces)	Backs off automatically if memory runs out, rather than dying hours in
Epochs	60 × 800 steps	About 48,000 steps, roughly 3.5 hours
Checkpointing	Every epoch	The laptop throttles; a crash costs one epoch

7. Making it fast

Half the score is wall-clock time on the judging machine, measured from process launch. This section covers what that changes.

7.1 What is actually being timed

```
[ program starts ]  <-- clock starts HERE
      |
      +-- import libraries              6.3 s
      +-- create GPU context, load model 2.8 s
      +-- read every input file          0.0 s  (hidden, see 7.3)
      +-- run the network                1.5 s
      +-- write every output file        0.03 s
      |
[ program ends ]    <-- clock stops HERE     TOTAL 10.7 s for 60 images
```

Note the shape of this: **fixed overhead dominates**. Only 1.5 of 10.7 seconds is the actual work. That is what makes start-up costs so consequential.

7.2 The optimisation that made things worse

GPU libraries offer an autotuning mode that tries several internal implementations and keeps the fastest. It is standard advice to switch it on. We measured it in fresh processes:

Setting	One-time warm-up	Per image	Total, 64 images
Autotuning **on**	**4,465 ms**	5.70 ms	4.739 s
Autotuning **off**	943 ms	5.93 ms	**1.228 s**

Autotuning spends 3.5 extra seconds to save 0.23 ms per image.

The break-even point is roughly **15,300 images**. The test set will not be remotely that large, so the standard advice is actively wrong here. Confirmed at the whole-program level: 10.7 seconds with it off, 15.9 seconds with it on.

The same logic applies to `torch.compile`, whose warm-up is larger still. Both remain available as options, off by default, with the measured break-even documented in the code so the decision can be revisited if the test set turns out to be enormous.

7.3 Free time from doing two things at once

Creating the GPU context takes about 2.8 seconds, during which the GPU sits idle. Rather than read files afterwards, every file read is dispatched to a pool of background threads **before** the GPU is touched. Reading then happens during that idle window.

Measured effect: time spent waiting for reads fell from 0.220 s to **0.000 s**. It is entirely hidden.

Threads rather than separate processes, deliberately: on Windows each worker process costs about a second to start, which on a run this short is pure loss. Image reading releases Python's global lock, so threads give the same overlap for no start-up cost.

7.4 Other measures

- **Images grouped by size before batching.** The test set mixes two resolutions; batching across sizes would force slow one-at-a-time processing.
- **Half precision throughout inference**, roughly doubling throughput.
- **Writes dispatched asynchronously** so disk output overlaps the next batch.
- **Heavy libraries never imported.** The perceptual-metric package alone downloads a 233 MB file on first use — harmless in training, catastrophic on a cold judging machine. It is not imported by the evaluation script at all.
- **Arguments validated before the expensive imports**, so a wrong path fails in 0.1 s rather than after 6 s of loading.
- **Optimiser state stripped from the shipped weights.** Training checkpoints carry roughly three times the necessary data; removing it cuts file size and load time.

Net result: **20.3 seconds down to 10.7 seconds**, a 47% reduction, with identical output.

8. Results

8.1 The three metrics, explained

Metric	Plain meaning	Direction
pSNR	Average pixel error, expressed in decibels. Every 6 dB is roughly a halving of error. Sensitive to overall accuracy, blind to whether the image *looks* right	higher better
SSIM	Compares local patterns, brightness and contrast rather than individual pixels. Ranges 0 to 1	higher better
LPIPS	Feeds both images through a pre-trained network and compares its internal responses. Correlates best with human judgement	**lower** better

8.2 The floor: what non-learned methods achieve

These were measured first, on the same held-out images. Their purpose is brutal: any trained model that fails to beat them is broken, and without them you would have no way to know.

Method	pSNR	SSIM	LPIPS
Bicubic enlargement only	19.924	0.5124	0.4709
Bicubic + median filter	21.395	**0.5560**	**0.4189**
Bicubic + bilateral filter	21.055	0.5547	0.4343
Median at small size, then bicubic	**21.802**	0.5214	0.4352

Baseline floors on 400 validation samples. An earlier run at 150 samples gave materially different numbers, which is why the larger sample is used throughout.

8.3 What the model achieves

Training completed: 60 epochs, best at epoch 54. Note that the SSIM in this curve (0.7294) is measured against this run's own degradation range, which is wider than the baselines were measured over. The comparison table below re-measures everything on one common footing and is the number to quote.

Epoch	pSNR	SSIM	LPIPS
0	22.995	0.6139	0.3732
1	23.492	0.6382	0.3512
2	23.907	0.6590	0.2849
4	24.325	0.6898	0.2604
15	24.789	0.7149	0.1994
30	24.978	0.7224	0.1822
45	25.132	0.7275	0.1755

Epoch	pSNR	SSIM	LPIPS
59	25.110	0.7288	0.1709
best (54)	**25.118**	**0.7294**	**0.1707**

Validation scores as training proceeds, on 525 held-out images.

Metric	Best baseline	Model	Improvement
pSNR (dB)	21.802	**24.420**	**+2.618**
SSIM	0.5560	**0.6925**	**+0.1365** (24.6%)
LPIPS	0.4189	**0.1511**	**-0.2678** (lower is better)

Model versus the strongest non-learned method on each metric.

8.4 End-to-end verification

Training scores are computed in memory. That is not the same as what the judges will measure, which is files on disk produced by our evaluation script. So the whole chain was tested by running the script and scoring its actual output files.

With an early, barely-trained model this already produced **+1.641 dB pSNR, +0.0554 SSIM and −0.0466 LPIPS** against bicubic — confirming that mixed input sizes, mixed file formats, output naming and output dimensions all behave correctly end to end.

8.5 The robustness sweep, and the second model it produced

The project's central risk is that the damage recipe was inferred and can never be confirmed. So the question that actually matters is not how the model scores at the assumed noise level, but **how fast it falls apart if that assumption is wrong**.

To measure it, the finished model was scored across a grid of noise strengths extending well beyond anything it trained on. The first model, trained over L = 8 to 40, gave this:

Speckle L	σ = 0	0.0086	0.05	note
4	0.4862	0.4788	0.4163	far below training range
8	0.6511	0.6491	0.5948	edge of training range
17	0.7042	0.7011	0.6335	the deck's actual setting
40	0.7511	0.7493	0.6613	edge of training range
100	0.7853	0.7827	0.6921	far above (easier)

SSIM of the first model across noise levels. Higher L means less noise.

Two things stand out. It degrades **gracefully upward** — less noise than expected simply scores better. But below L = 8 it falls away sharply, losing 0.17 SSIM between L = 8 and L = 4.

So a second model was trained: wider network (19.6M parameters instead of 11.1M) and a wider damage range (L from 5 to 60). Scored on the identical grid:

Speckle L	first model	second model	gain
4, σ=0.05	0.4163	**0.5620**	**+0.1457**
4, σ=0	0.4862	**0.5970**	**+0.1108**
8, σ=0.0086	0.6491	**0.6522**	+0.0031
17, σ=0.0086	0.7011	**0.7034**	+0.0023
100, σ=0	0.7853	**0.7933**	+0.0080

The second model wins in every cell of the grid, by far the most where the first was weakest. Worst-case degradation fell from 40.5% to 20.0%.

Why the second model ships

It wins on **every** metric on identical data (see 8.3), it roughly halves the worst-case collapse outside the trained range, and it costs **no measurable inference time** — 4.913 s versus 4.855 s on the same 60 images, which is run-to-run noise.

The last point is not obvious: it has 78% more parameters. At these batch sizes the work is limited by memory bandwidth rather than arithmetic, so the extra capacity is close to free.

8.6 Robustness checks

The evaluation script was deliberately attacked with awkward inputs, because a crash scores zero for everything remaining, which is far worse than a slightly lower score.

Case	Result
Single image	passes
Non-square image (120×200)	passes
Odd sizes not divisible by the network's scale factor	passes, exact 2× output
Colour input instead of greyscale	passes
Mixed formats and bit depths in one folder	passes
A corrupt file among valid ones	**skipped, neighbours still processed**
Images in nested sub-folders	passes
Empty input folder	clean error message, no crash
Image larger than the stated maximum	passes

9. What we verified, and what we cannot

9.1 The verification discipline

Because no real data existed, ordinary sanity checks were unavailable. The practice adopted throughout was: **construct a situation where the answer is known, then check the code finds it.**

- Measurement code was tested against synthetic data with planted parameters (recovered to 1.4% and 0.2%).
- The metrics code was tested by feeding it progressively noisier images and confirming the scores move in the right direction, and that an image scored against itself gives a perfect result.
- The network was tested at both required sizes and at deliberately awkward sizes.
- The dataset loader was tested on a purpose-built fake dataset with mixed formats and decoy filenames.
- The evaluation script was tested from a different working directory and against malformed inputs.

9.2 Real defects this caught

These were all found by testing rather than by reading the code, and every one would have surfaced later at a worse moment.

Defect	Consequence had it survived
A removed NumPy function still being called	Immediate crash the first time real analysis was run
Filename matching using loose substring search	A file named `train_0042.png` was silently classified as damaged. Every file would have been mislabelled and the tool would have reported zero pairs while appearing to work
Brightness normalised using the wrong image's format	Would have destroyed the single most important measurement in the analysis
Truncated downloads saved as if complete	A half-downloaded file cached as valid, corrupting the dataset silently
Blurring before comparing shrinking methods	The identification test could not identify its own planted answer
Additive noise estimated by extrapolation	35% error, and impossible negative values on individual images
Too many loading processes for available memory	Training crashed on start-up; now falls back automatically instead

A note on passing tests

The verification suite reported **PASS** on its first run — while the shrinking-method test was completely unable to identify its own answer and the noise estimate was 35% wrong. The thresholds were simply loose enough to let both through.

A green tick is not evidence. The numbers behind it are.

9.3 What remains genuinely uncertain

Stated plainly, because these are the real risks:

1. **The damage model is inferred, not confirmed.** It comes from two figure captions and reproduces the published behaviour closely, but with no real paired data it can never be verified. If it is materially wrong, everything downstream inherits the error. The deliberately wide training ranges are the insurance.
2. **The order of the two noises is assumed.** We apply speckle and then additive noise. The reverse order is possible and no test distinguishes them.
3. **Whether speckle is applied before or after shrinking is only partly settled.** Our test covers most cases but has a blind spot for one specific combination.
4. **The expected output file format is a reasonable guess.** We mirror the input's format and filename. If the judges expect something else, quality becomes irrelevant — this is the highest-impact unknown remaining.
5. **The test set size is unknown**, and it changes the speed calculus. Our measurements assume a set far below 15,000 images.

9.4 Done since the first draft

- **The sensitivity sweep** was run (section 8.5). It found a real weakness below L = 8 and directly motivated the second training run.
- **A wider network** was trained, combined with the wider damage range. It wins on every metric and every grid cell, at no inference cost, and is what ships.

9.5 What would be done next, with more time

- **Test-time averaging** — run each image through in eight rotations and average. Reliably worth a few tenths of a decibel, but costs eight times the inference, so it must be weighed against the speed score rather than assumed beneficial.
- **Loss weight tuning.** The 1.0 / 0.2 / 0.1 split is a sensible starting point that has never been optimised.
- **Wider still.** The second model beat the first at no speed cost, which suggests the capacity ceiling has not been reached. A third run at greater width is the obvious next experiment.
- **More ground-truth sources.** Everything the model knows comes from three corpora of our choosing, and generalisation is explicitly scored.

10. Glossary

Term	Meaning
Additive noise	Random amounts added to pixels, the same magnitude regardless of brightness. Ordinary 'static'.
AMP / mixed precision	Doing most arithmetic in half precision for speed while keeping delicate steps in full precision.
Batch	A group of images processed together in one step, for efficiency.
Bicubic interpolation	A standard way of enlarging an image by fitting smooth curves through existing pixels.
Channel	One of many parallel 'views' of an image inside a network, each highlighting a different feature.
Charbonnier loss	A pixel-difference measure that behaves like squared error for small differences and like absolute error for large ones, so extreme outliers do not dominate.
Checkpoint	A saved snapshot of the model, so training can resume after a crash.
Convolution	Sliding a small window of learned numbers across an image; the fundamental operation of these networks.
Denoising	Removing noise from an image.
Epoch	One pass through a defined amount of training data.
Gamma distribution	The statistical distribution the speckle multiplier is drawn from. Here it has average 1 and spread $1/\sqrt{L}$.
Ground truth	The correct answer — the original undamaged image.
L (number of looks)	The speckle strength parameter. Higher means less noise; relative noise is $1/\sqrt{L}$.
LPIPS	A perceptual similarity score computed by comparing two images through a pre-trained network. Lower is better.
Multiplicative noise	Noise that scales each pixel rather than adding to it, so bright areas are damaged more than dark ones.
NAFNet	The network architecture used here; notable for using no activation functions and no attention.
Overfitting	When a model memorises its training examples instead of learning general rules, and so performs poorly on anything new.
Parameter	One of the adjustable numbers inside a network. This model has about 11 million.
PixelShuffle	Enlarging an image by rearranging channels into space, with no arithmetic and therefore no artifacts.
pSNR	Peak signal-to-noise ratio; average pixel error in decibels. Higher is better.
Residual / skip connection	Adding a layer's input to its output, so the layer only needs to learn a correction.
Sigma (σ)	The spread of the additive noise.

Term	Meaning
Speckle	Grainy multiplicative noise inherent to coherent imaging such as radar, ultrasound and some microscopy.
SSIM	Structural similarity index; compares local patterns and contrast rather than individual pixels. Higher is better, maximum 1.
Super-resolution	Producing a larger, more detailed image from a smaller one.
Validation set	Images held out of training, used to measure honest performance on material the model has not seen.